

Final Lab Task Submission

Artificial Intelligence — Final Lab Task

Name: Minhazul Abedin

ID: _____

Section: _____

Semester: Summer 25-26

Task 1 — Goal-Based Agent: Automated Water Purification System

The agent reads the water sensors (Turbidity, TDS, Bacteria, pH) and keeps applying **condition-action rules** — Sedimentation, Filtration, Reverse Osmosis, UV Treatment, Chemical Addition — printing the updated state after every action, until the goal state (Turbidity ≤ 1 , TDS ≤ 300 , Bacteria = False, pH 6.5–8.5) is reached.

Code Screenshot

```
task1_water_purification_agent.py - AI Lab - Visual Studio Code

task1_water_purification_agent.py x

1  # Final Lab Task 1
2  # Goal-Based Agent : Automated Water Purification System
3
4
5  def show(w):
6      return (f"Turbidity = {w['Turbidity']}, TDS = {w['TDS']}, "
7              f"Bacteria = {w['Bacteria']}, pH = {w['pH']}")
8
9
10 def goal_reached(w):
11     if w["Turbidity"] <= 1 and w["TDS"] <= 300 and w["Bacteria"] == False:
12         if 6.5 <= w["pH"] <= 8.5:
13             return True
14     return False
15
16
17 # sensor readings of the incoming water
18 water = {"Turbidity": 5.0, "TDS": 700, "Bacteria": True, "pH": 5.8}
19
20 print("Initial water quality:", show(water))
21
22 # keep applying the rules until the goal state is reached
23 while not goal_reached(water):
24
25     if water["Turbidity"] > 3:
26         print("→ Turbidity > 1 → Action Taken: Sedimentation")
27         water["Turbidity"] = round(water["Turbidity"] * 0.1, 2)
28
29     elif water["Turbidity"] > 1:
30         print("→ Turbidity > 1 → Action Taken: Filtration")
31         water["Turbidity"] = round(water["Turbidity"] * 0.4, 2)
32
33     elif water["TDS"] > 300:
34         print("→ TDS > 300 → Action Taken: Reverse Osmosis")
35         water["TDS"] = int(water["TDS"] * 0.4)
36
```

Ln 36, Col 1 Spaces: 4 UTF-8 LF Python 3.11.4 64-bit

task1_water_purification_agent.py ×

```
37     elif water["Bacteria"] == True:
38         print("→ Bacteria = True → Action Taken: UV Treatment")
39         water["Bacteria"] = False
40
41     elif water["pH"] < 6.5:
42         print("→ pH < 6.5 → Action Taken: Add Chemicals")
43         water["pH"] = 7.2
44
45     elif water["pH"] > 8.5:
46         print("→ pH > 8.5 → Action Taken: Add Chemicals")
47         water["pH"] = 7.2
48
49     print("    Updated state:", show(water))
50
51     print("Water quality after treatment:", show(water))
52     print("Goal Achieved: Water is purified and safe for drinking.")
```

0 0 0

Ln 52, Col 1 Spaces: 4 UTF-8 LF Python 3.11.4 64-bit

Output Screenshot

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Minhaz\Desktop\AI Lab> python task1_water_purification_agent.py
Initial water quality: Turbidity = 5.0, TDS = 700, Bacteria = True, pH = 5.8
→ Turbidity > 1 → Action Taken: Sedimentation
    Updated state: Turbidity = 0.5, TDS = 700, Bacteria = True, pH = 5.8
→ TDS > 300 → Action Taken: Reverse Osmosis
    Updated state: Turbidity = 0.5, TDS = 280, Bacteria = True, pH = 5.8
→ Bacteria = True → Action Taken: UV Treatment
    Updated state: Turbidity = 0.5, TDS = 280, Bacteria = False, pH = 5.8
→ pH < 6.5 → Action Taken: Add Chemicals
    Updated state: Turbidity = 0.5, TDS = 280, Bacteria = False, pH = 7.2
Water quality after treatment: Turbidity = 0.5, TDS = 280, Bacteria = False, pH = 7.2
Goal Achieved: Water is purified and safe for drinking.
```

Task 2 — Simple Rule-Based Vacuum Cleaning Agent

In a 4×4 grid world (**D** = dirty, **.** = clean, **A** = agent) the agent **perceives** whether its current cell is dirty, **decides** its action (clean, move right, or move down to the next row) and **executes** it, printing the environment after every action until every cell is clean.

Code Screenshot

```
task2_vacuum_cleaning_agent.py - AI Lab - Visual Studio Code

task2_vacuum_cleaning_agent.py ×

1  # Final Lab Task 2
2  # Simple Rule-Based Vacuum Cleaning Agent
3
4  rows = 4
5  cols = 4
6
7  # 'D' = dirty cell , '.' = clean cell
8  grid = [
9      ['D', '.', 'D', '.'],
10     ['D', '.', 'D', '.'],
11     ['D', '.', '.', '.'],
12     ['D', 'D', 'D', 'D']
13 ]
14
15
16 def show_grid(r, c):
17     for i in range(rows):
18         for j in range(cols):
19             if i == r and j == c:
20                 print('A', end=' ')      # A = agent position
21             else:
22                 print(grid[i][j], end=' ')
23         print()
24     print('-----')
25
26
27 row = 0
28 col = 0
29 count = 0
30
31 print("Initial Environment:")
32 show_grid(row, col)
33
```

Ln 33, Col 1 Spaces: 4 UTF-8 LF Python 3.11.4 64-bit

task2_vacuum_cleaning_agent.py ×

```
34 while True:
35
36     # rule 1 : if the current cell is dirty then clean it
37     if grid[row][col] == 'D':
38         grid[row][col] = '.'
39         count = count + 1
40         print(f"Cleaned cell at ({row}, {col})")
41         show_grid(row, col)
42
43     # rule 2 : otherwise move to the next cell
44     if col < cols - 1:
45         col = col + 1                # move right
46     elif row < rows - 1:
47         row = row + 1                # move down
48         col = 0
49     else:
50         break                        # whole grid is scanned
51
52     show_grid(row, col)
53
54 show_grid(row, col)
55 print("Cleaning complete. Total cells cleaned:", count)
```

Output Screenshot

Console output, captured in three parts (scrolled).

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Minhaz\Desktop\AI Lab> python task2_vacuum_cleaning_agent.py
Initial Environment:
A . D .
D . D .
D . . .
D D D D
-----
Cleaned cell at (0, 0)
A . D .
D . D .
D . . .
D D D D
-----
. A D .
D . D .
D . . .
D D D D
-----
. . A .
D . D .
D . . .
D D D D
-----
Cleaned cell at (0, 2)
. . A .
D . D .
D . . .
D D D D
-----
. . . A
D . D .
D . . .
D D D D
-----
. . . .
A . D .
D . . .
D D D D
-----
Cleaned cell at (1, 0)
. . . .
A . D .
D . . .
D D D D
-----
. . . .
. A D .
D . . .
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . A .
```

```
D . . .
```

```
D D D D
```

```
-----
```

```
Cleaned cell at (1, 2)
```

```
. . . .
```

```
. . A .
```

```
D . . .
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . . A
```

```
D . . .
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
A . . .
```

```
D D D D
```

```
-----
```

```
Cleaned cell at (2, 0)
```

```
. . . .
```

```
. . . .
```

```
A . . .
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. A . .
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . A .
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . . A
```

```
D D D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
A D D D
```

```
-----
```

```
Cleaned cell at (3, 0)
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
A D D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. A D D
```

```
-----
```

```
Cleaned cell at (3, 1)
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. A D D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. . A D
```

```
-----
```

```
Cleaned cell at (3, 2)
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. . A D
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. . . A
```

```
-----
```

```
Cleaned cell at (3, 3)
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. . . A
```

```
-----
```

```
. . . .
```

```
. . . .
```

```
. . . .
```

```
. . . A
```

```
-----
```

```
Cleaning complete. Total cells cleaned: 9
```